Simple Model

```r
suppressPackageStartupMessages(library(tidyverse))
suppressPackageStartupMessages(library(ggplot2))
suppressPackageStartupMessages(library(dplyr))
suppressPackageStartupMessages(library(bayesplot))
suppressPackageStartupMessages(library(cmdstanr))
```

```r
# ========= load and preprocess data ================
data <- read.csv("full_data.csv")
colnames(data)
```

```
##  [1] "driver_id"      "constructor_id" "points"         "position"
##  [5] "grid"           "laps"           "status"         "gap"
##  [9] "fastest_rank"   "fastest"        "top_speed_kph"  "time_sec"
## [13] "season"         "round"          "race_name"      "circuit"
## [17] "gap_seconds"
```

```r
head(data)
```

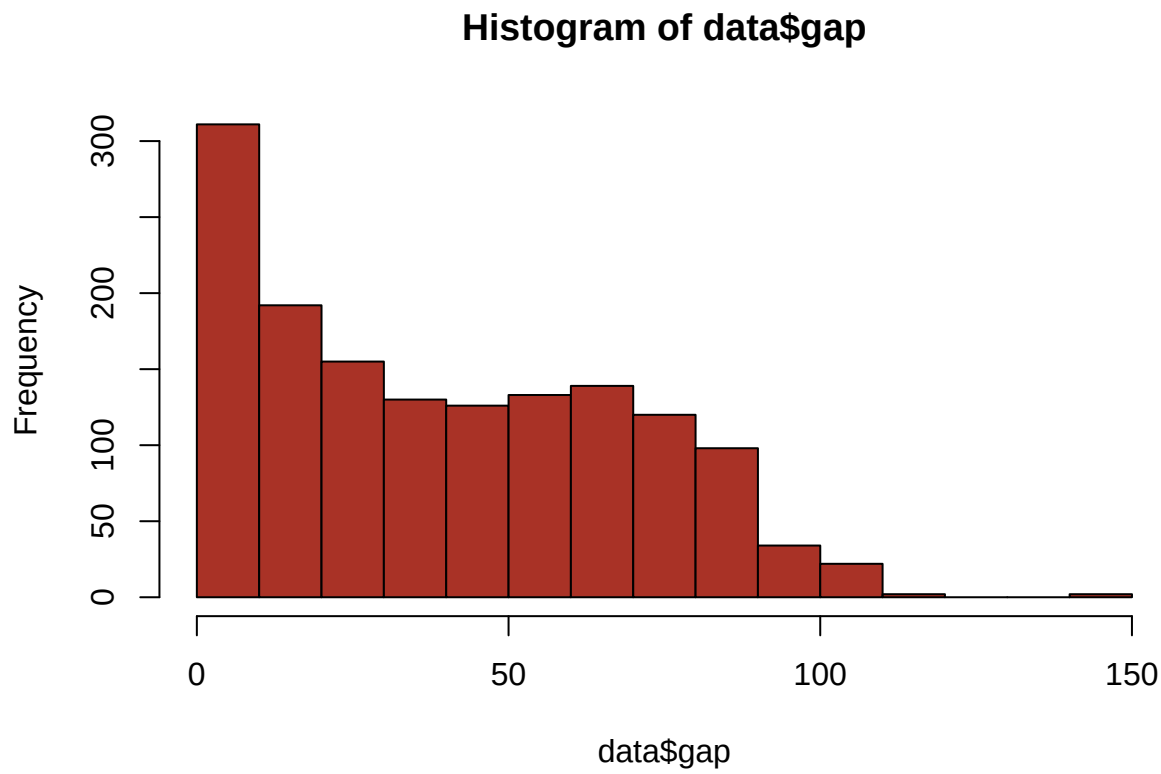
```
##         driver_id constructor_id points position grid laps   status         gap
## 1          bottas       mercedes     26        1    2   58 Finished 1:25:27.325
## 2        hamilton       mercedes     18        2    1   58 Finished     +20.886
## 3  max_verstappen       red_bull     15        3    4   58 Finished     +22.520
## 4          vettel        ferrari     12        4    3   58 Finished     +57.109
## 5         leclerc        ferrari     10        5    5   58 Finished     +58.203
## 6 kevin_magnussen           haas      8        6    7   58 Finished   +1:27.156
##   fastest_rank  fastest top_speed_kph time_sec season round
## 1            1 1:25.580       223.075   85.580   2019     1
## 2            2 1:26.057       221.839   86.057   2019     1
## 3            3 1:26.256       221.327   86.256   2019     1
## 4            8 1:27.954       217.054   87.954   2019     1
## 5            4 1:26.926       219.621   86.926   2019     1
## 6            9 1:28.182       216.493   88.182   2019     1
##               race_name     circuit gap_seconds
## 1 Australian Grand Prix albert_park       0.000
## 2 Australian Grand Prix albert_park      20.886
## 3 Australian Grand Prix albert_park      22.520
## 4 Australian Grand Prix albert_park      57.109
## 5 Australian Grand Prix albert_park      58.203
## 6 Australian Grand Prix albert_park      87.156
```

```r
convert_to_seconds <- function(x) {
  if (is.numeric(x)) return(x)

  x <- as.character(x)

  sapply(x, function(val) {
    # Plain number (no colon)
    if (grepl("^[+-]?\\d+\\.?\\d*$", val)) {
```

```r
      return(as.numeric(val))
    }

    # No sign prefix and contains a colon = gap of 0
    if (!grepl("^[+-]", val) && grepl(":", val)) {
      return(0)
    }

    # Extract sign
    sign <- ifelse(startsWith(val, "-"), -1, 1)
    val <- gsub("^[+-]", "", val)

    parts <- strsplit(val, ":")[[1]]

    if (length(parts) == 2) {
      minutes <- as.numeric(parts[1])
      seconds <- as.numeric(parts[2])
      return(sign * (minutes * 60 + seconds))
    } else if (length(parts) == 3) {
      hours   <- as.numeric(parts[1])
      minutes <- as.numeric(parts[2])
      seconds <- as.numeric(parts[3])
      return(sign * (hours * 3600 + minutes * 60 + seconds))
    } else {
      return(NA)
    }
  }, USE.NAMES = FALSE)
}
```

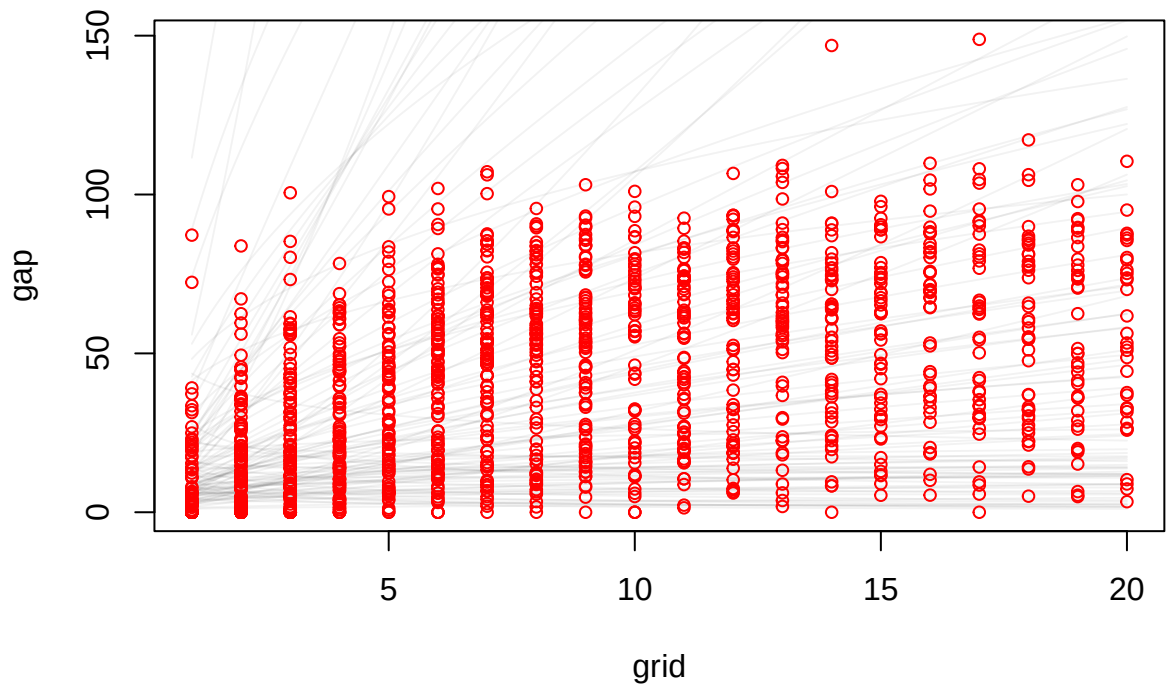
```r
data <- data %>%
  mutate(gap = convert_to_seconds(gap),
         grid = as.integer(grid)) %>%
  filter(
    status == "Finished",       # only finishers
    !is.na(grid),
    grid > 0,                    # exclude pit lane starts (grid = 0)
    !is.na(gap))

set.seed(1)
```

```r
# ========= prior check ================
# distribution of data
hist(data$gap, col = "#a93226")
```




```r
n_samples <- 100
xs <- seq(1, 20, by = 0.1)

# sample from priors
# alpha_prior <- rnorm(n_samples, 0, 5)
# beta_prior  <- rnorm(n_samples, 0, 2)

alpha_prior <- rnorm(n_samples, -2.5, 1.0)
beta_prior  <- rnorm(n_samples, -0.5, 0.5)
```

```r
# set up empty plot with range of your actual data
plot(NULL, xlim = c(1, 20), ylim = range(data$gap),
     xlab = "grid", ylab = "gap")

# draw prior curves first
for (i in 1:n_samples) {
  mean_gap <- exp(-(alpha_prior[i] + beta_prior[i] * log(xs)))
  lines(xs, mean_gap, col = rgb(0, 0, 0, alpha = 0.05))
}

# overlay actual data on top
points(data$grid, data$gap, col = "red", pch = 1, cex = 0.8)
```




```
# ========= running stan ================
# create directory for stan file
dir.create(file.path("stan_out"), showWarnings = FALSE)

grid_pred_pos <- 10

N_obs = nrow(data)

# exponential model
mod_simple_exp = cmdstan_model("simple_model_exp.stan")
fit_simple_exp = mod_simple_exp$sample(
  seed = 1,
  chains = 5,
  refresh = 0,
  output_dir = "stan_out",
  data = list(N = N_obs,
              grid = data$grid,
              gap = data$gap,
              grid_pred = grid_pred_pos),
  iter_warmup = 500,
  iter_sampling = 500,
)
```

```
grid_pred_pos <- 10

fit_exp.variational = mod_simple_exp$variational(
  seed = 1,
```

```r
  refresh = 0,
  show_messages = FALSE,
  show_exceptions = FALSE,
  output_dir = "stan_out",
  algorithm = "meanfield",
  output_samples = 1000,
  data = list(N = N_obs,
              grid = data$grid,
              gap = data$gap,
              grid_pred = grid_pred_pos)
)
```

```r
# ========= model summary (ESS and R hat) ================
print(fit_simple_exp)
```

```
##     variable     mean   median   sd  mad       q5      q95 rhat ess_bulk
##  lp__        -6651.15 -6650.86 0.94 0.71 -6653.00 -6650.24 1.01     1000
##  alpha          -2.26    -2.26 0.07 0.07    -2.37    -2.15 1.01      559
##  beta           -0.70    -0.70 0.03 0.03    -0.75    -0.64 1.01      545
##  log_grid[1]     0.69     0.69 0.00 0.00     0.69     0.69   NA       NA
##  log_grid[2]     0.00     0.00 0.00 0.00     0.00     0.00   NA       NA
##  log_grid[3]     1.39     1.39 0.00 0.00     1.39     1.39   NA       NA
##  log_grid[4]     1.10     1.10 0.00 0.00     1.10     1.10   NA       NA
##  log_grid[5]     1.61     1.61 0.00 0.00     1.61     1.61   NA       NA
##  log_grid[6]     1.95     1.95 0.00 0.00     1.95     1.95   NA       NA
##  log_grid[7]     1.10     1.10 0.00 0.00     1.10     1.10   NA       NA
##  ess_tail
##      1410
##       739
##       835
##        NA
##        NA
##        NA
##        NA
##        NA
##        NA
##        NA
##
##  # showing 10 of 1469 rows (change via 'max_rows' argument or 'cmdstanr_max_rows' option)
```

```r
fit_simple_exp$summary(NULL, c("mean", "sd", "mcse_mean", "ess_bulk"))
```

```
## # A tibble: 1,469 x 5
##    variable         mean     sd mcse_mean ess_bulk
##    <chr>           <dbl>  <dbl>     <dbl>    <dbl>
##  1 lp__        -6651.    0.944    0.0310     1000.
##  2 alpha          -2.26  0.0651   0.00276     560.
##  3 beta           -0.697 0.0329   0.00141     545.
##  4 log_grid[1]     0.693 0       NA            NA
##  5 log_grid[2]     0     0       NA            NA
##  6 log_grid[3]     1.39  0       NA            NA
##  7 log_grid[4]     1.10  0       NA            NA
##  8 log_grid[5]     1.61  0       NA            NA
##  9 log_grid[6]     1.95  0       NA            NA
## 10 log_grid[7]     1.10  0       NA            NA
```

```
## # i 1,459 more rows
```

```r
print(fit_exp.variational)
```

```
##     variable     mean   median   sd  mad       q5      q95
##  lp__        -6660.41 -6659.53 5.59 5.37 -6670.53 -6653.20
##  lp_approx__    -0.99    -0.65 1.01 0.68    -3.09    -0.05
##  alpha          -2.22    -2.22 0.02 0.02    -2.25    -2.18
##  beta           -0.66    -0.66 0.01 0.01    -0.69    -0.64
##  log_grid[1]     0.69     0.69 0.00 0.00     0.69     0.69
##  log_grid[2]     0.00     0.00 0.00 0.00     0.00     0.00
##  log_grid[3]     1.39     1.39 0.00 0.00     1.39     1.39
##  log_grid[4]     1.10     1.10 0.00 0.00     1.10     1.10
##  log_grid[5]     1.61     1.61 0.00 0.00     1.61     1.61
##  log_grid[6]     1.95     1.95 0.00 0.00     1.95     1.95
##
##  # showing 10 of 1470 rows (change via 'max_rows' argument or 'cmdstanr_max_rows' option)
```

```r
fit_exp.variational$summary(NULL, c("mean", "sd", "mcse_mean", "ess_bulk"))
```

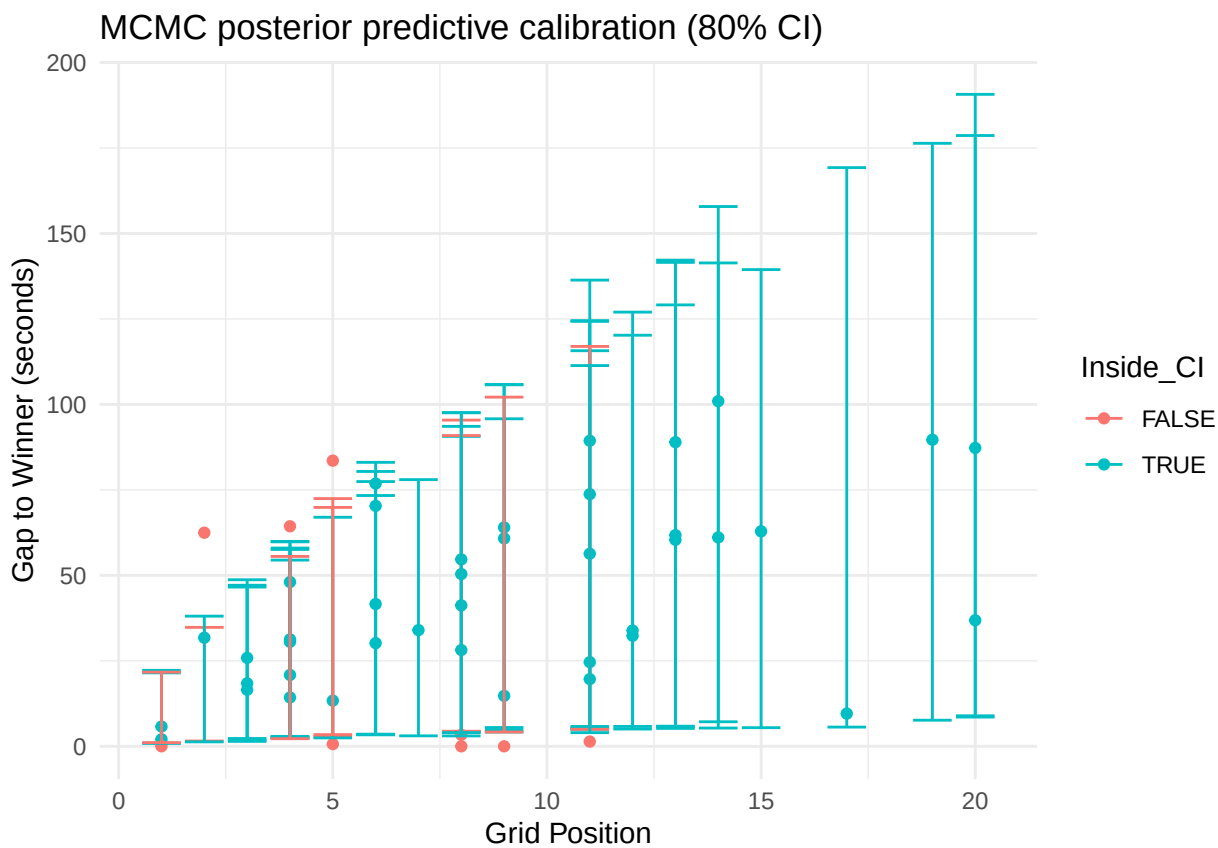
```
## # A tibble: 1,470 x 5
##    variable         mean     sd mcse_mean ess_bulk
##    <chr>           <dbl>  <dbl>     <dbl>    <dbl>
##  1 lp__        -6660.    5.59    0.168       1101.
##  2 lp_approx__    -0.993 1.01    0.0325       972.
##  3 alpha          -2.22  0.0203  0.000649     975.
##  4 beta           -0.664 0.0139  0.000411    1142.
##  5 log_grid[1]     0.693 0      NA             NA
##  6 log_grid[2]     0     0      NA             NA
##  7 log_grid[3]     1.39  0      NA             NA
##  8 log_grid[4]     1.10  0      NA             NA
##  9 log_grid[5]     1.61  0      NA             NA
## 10 log_grid[6]     1.95  0      NA             NA
## # i 1,460 more rows
```

```r
# TODO change this to simple cross validation
# ========= calibration ================
set.seed(1)
N_train = N_obs - 1

ci_level = 0.8
ci_plims = c((1 - ci_level) / 2, (1 + ci_level) / 2)

# randomly sample 50 points from the data and set that as the validation since
# there are over 50 data points TODO is this a valid method?
validation_obs <- sample(1:N_obs, 50, replace = FALSE)
```

```r
# sample 50 to save time
all_mcmc_samples <- list()
ci_limits_exp = matrix(0, nrow = 50, 2)

for (i in 1:length(validation_obs)) {
  i_test = validation_obs[i]
  train_test_dta = list(
    N = N_train,
    grid = data$grid[-i_test],
```

```r
    gap = data$gap[-i_test],
    grid_pred = data$grid[i_test]
  )
  mcmc_results = mod_simple_exp$sample(
    seed = 1,
    data = train_test_dta,
    output_dir = "stan_out",
    refresh = 0,
    show_messages = FALSE,
    show_exceptions = FALSE,
    chains = 1,
    iter_warmup = 500,
    iter_sampling = 500
  )
  simple_exp_samples = as.vector(mcmc_results$draws("gap_pred"))
  all_mcmc_samples[[i]] <- as.vector(mcmc_results$draws("gap_pred"))
  ci_limits_exp[i, ] = quantile(simple_exp_samples, ci_plims)
}
```

```r
# MCMC Calibration plot

merged_data_exp = data[validation_obs, ] %>%
  bind_cols(data.frame(CI_L = ci_limits_exp[, 1], CI_R = ci_limits_exp[, 2])) %>%
  mutate(Inside_CI = (gap >= CI_L & gap <= CI_R))

merged_data_exp %>%
  ggplot(aes(x = grid, y = gap, ymin = CI_L, ymax = CI_R, color = Inside_CI)) +
  geom_point() +
  geom_errorbar() +
  theme_minimal() +
  labs(title = "MCMC posterior predictive calibration (80% CI)",
     x = "Grid Position", y = "Gap to Winner (seconds)")
```



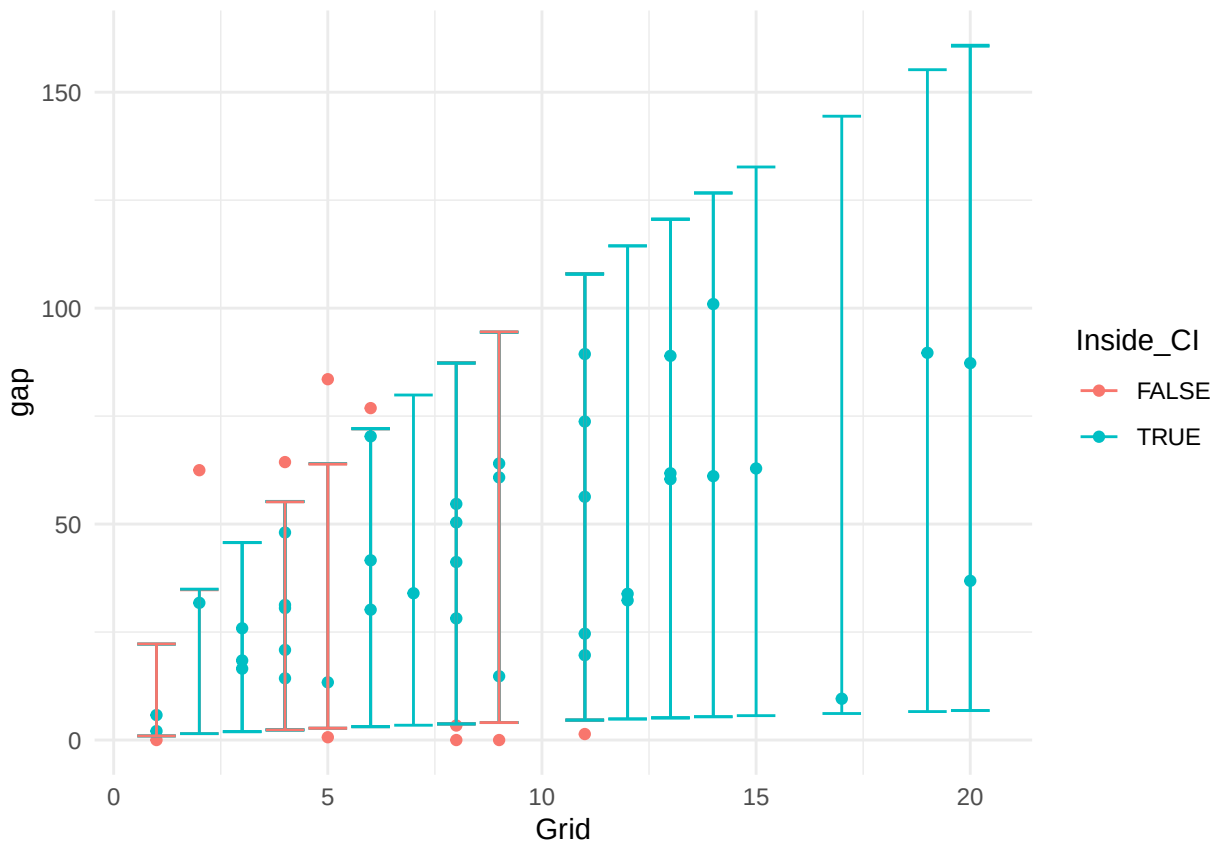

```r
mean(merged_data_exp$Inside_CI)
```

```
## [1] 0.82
```

```r
all_vi_samples <- list()
ci_limits_exp_var = matrix(0, nrow = 50, 2)

for (i in 1:length(validation_obs)) {
  i_test = validation_obs[i]
  train_test_dta = list(
    N = N_train,
    grid = data$grid[-i_test],
    gap = data$gap[-i_test],
    grid_pred = data$grid[i_test]
  )
  vi_results = mod_simple_exp$variational(
    seed = 1,
    data = train_test_dta,
    output_dir = "stan_out",
    refresh = 0,
    show_messages = FALSE,
    show_exceptions = FALSE,
    algorithm = "meanfield",
    output_samples = 1000,
  )
  simple_exp_var_samples = as.vector(vi_results$draws("gap_pred"))
  all_vi_samples[[i]] <- as.vector(vi_results$draws("gap_pred"))
```

```r
  ci_limits_exp_var[i, ] = quantile(simple_exp_var_samples, ci_plims)
}
```

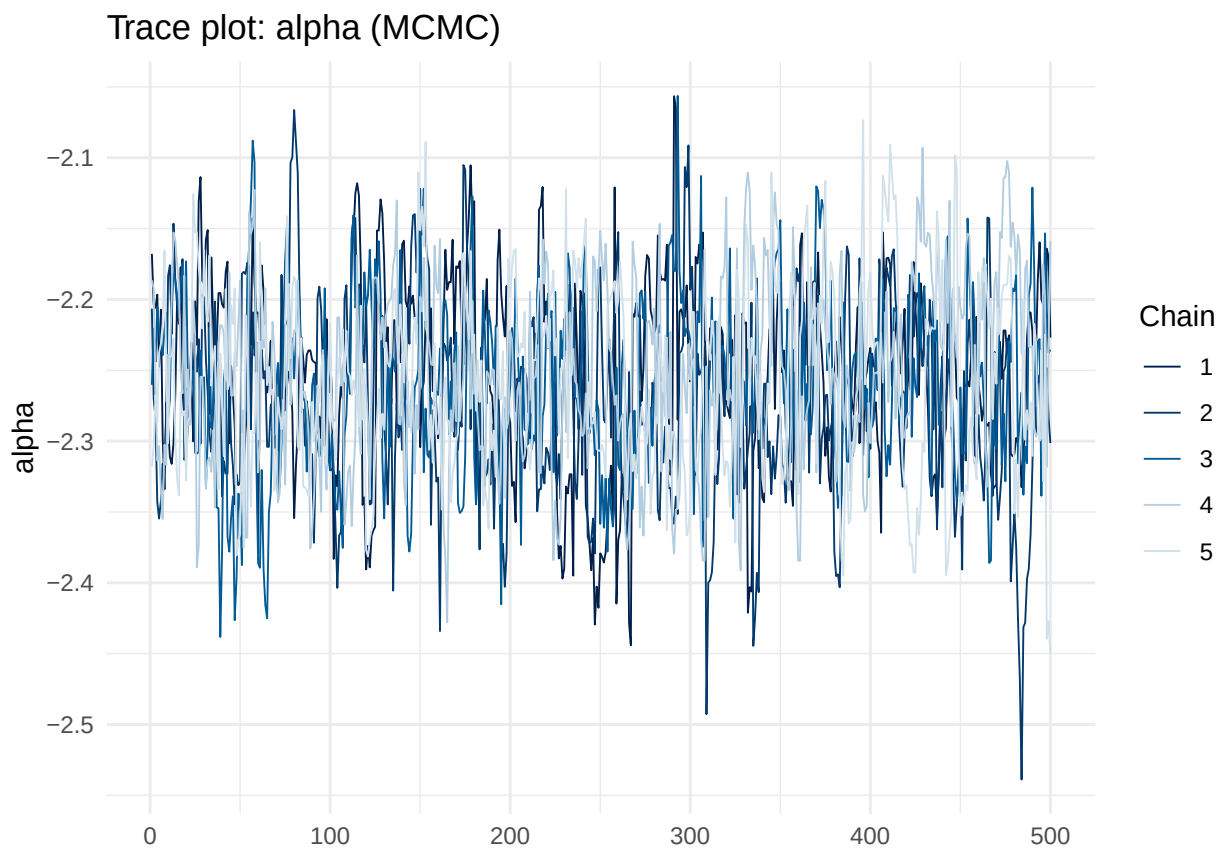
```r
merged_data_exp_var = data[validation_obs, ] %>%
  bind_cols(data.frame(CI_L = ci_limits_exp_var[, 1], CI_R = ci_limits_exp_var[, 2])) %>%
  mutate(Inside_CI = (gap >= CI_L & gap <= CI_R))

merged_data_exp_var %>%
  ggplot(aes(x = grid, y = gap, ymin = CI_L, ymax = CI_R, color = Inside_CI)) +
  geom_point() +
  geom_errorbar() +
  theme_minimal() +
  labs(x = "Grid", y = "gap")
```



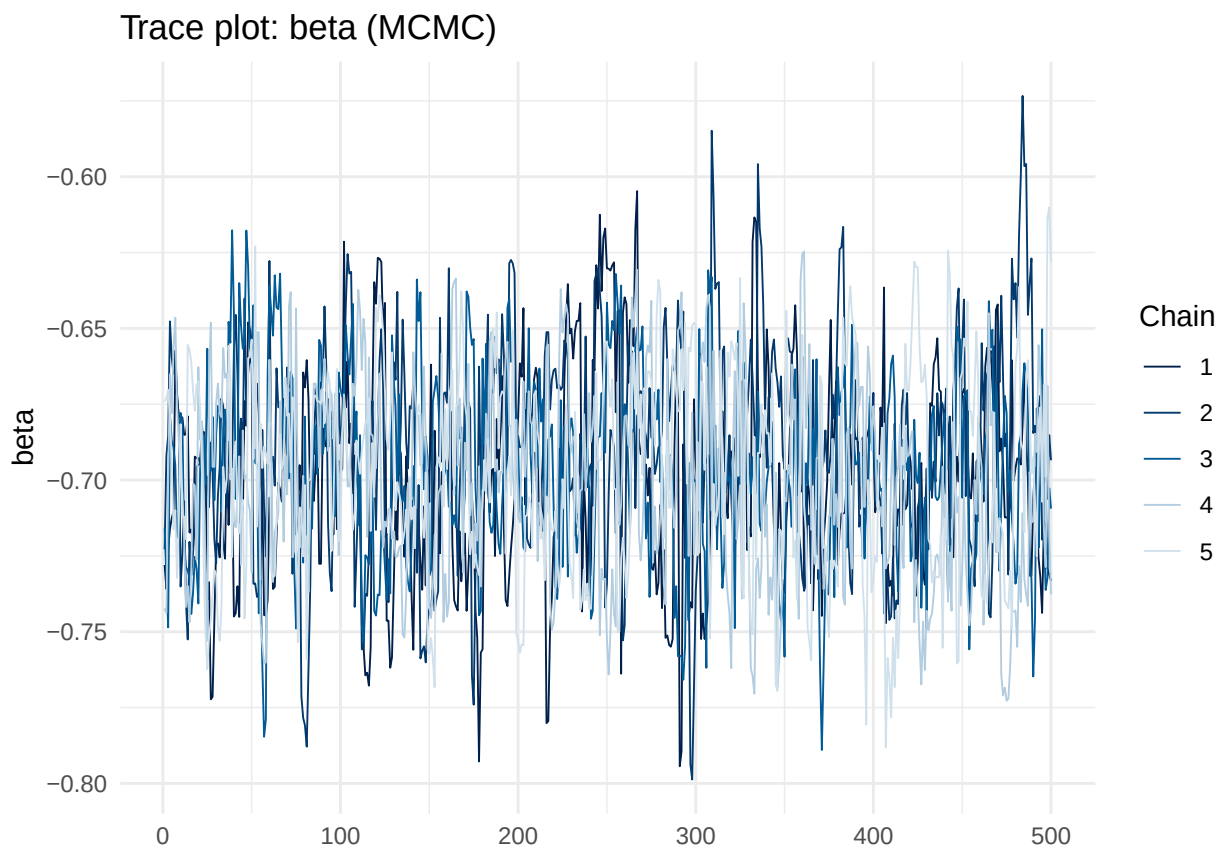

```r
mean(merged_data_exp_var$Inside_CI)
```

```
## [1] 0.8
```

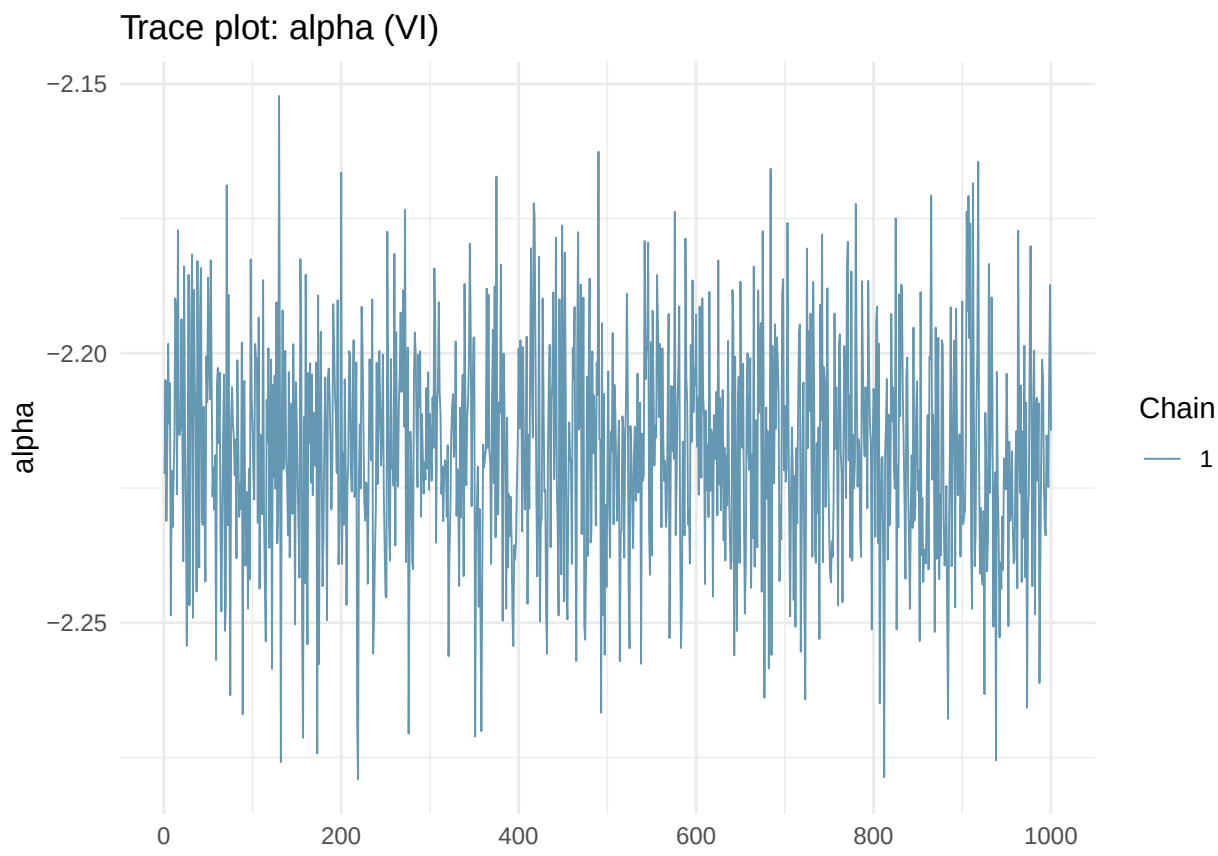
```r
# ========= mixing and predictive posterior ================
# trace plots for MH and VI
mcmc_trace(fit_simple_exp$draws("alpha")) +
  theme_minimal() +
  labs(title = "Trace plot: alpha (MCMC)")
```




```r
mcmc_trace(fit_simple_exp$draws("beta")) +
  theme_minimal() +
  labs(title = "Trace plot: beta (MCMC)")
```

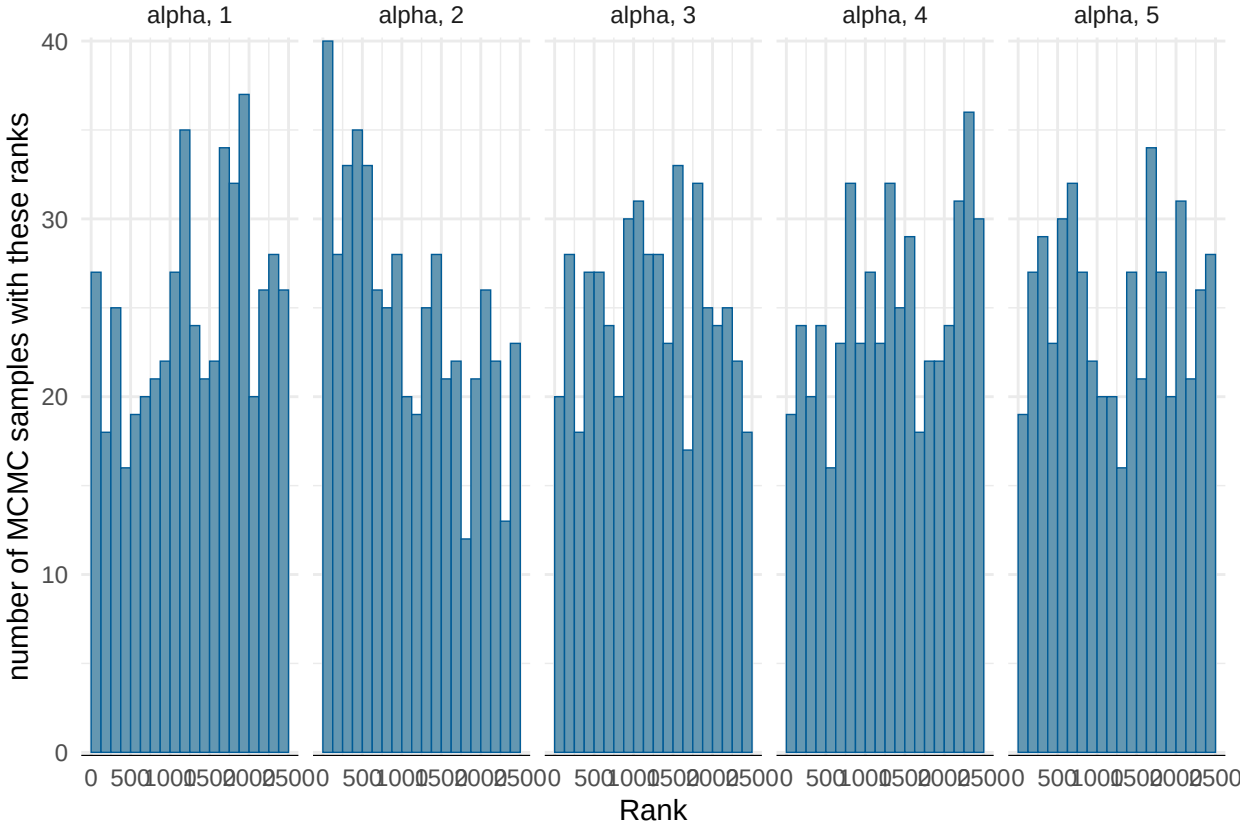



```r
mcmc_trace(fit_exp.variational$draws("alpha")) +
  theme_minimal() +
  labs(title = "Trace plot: alpha (VI)")
```

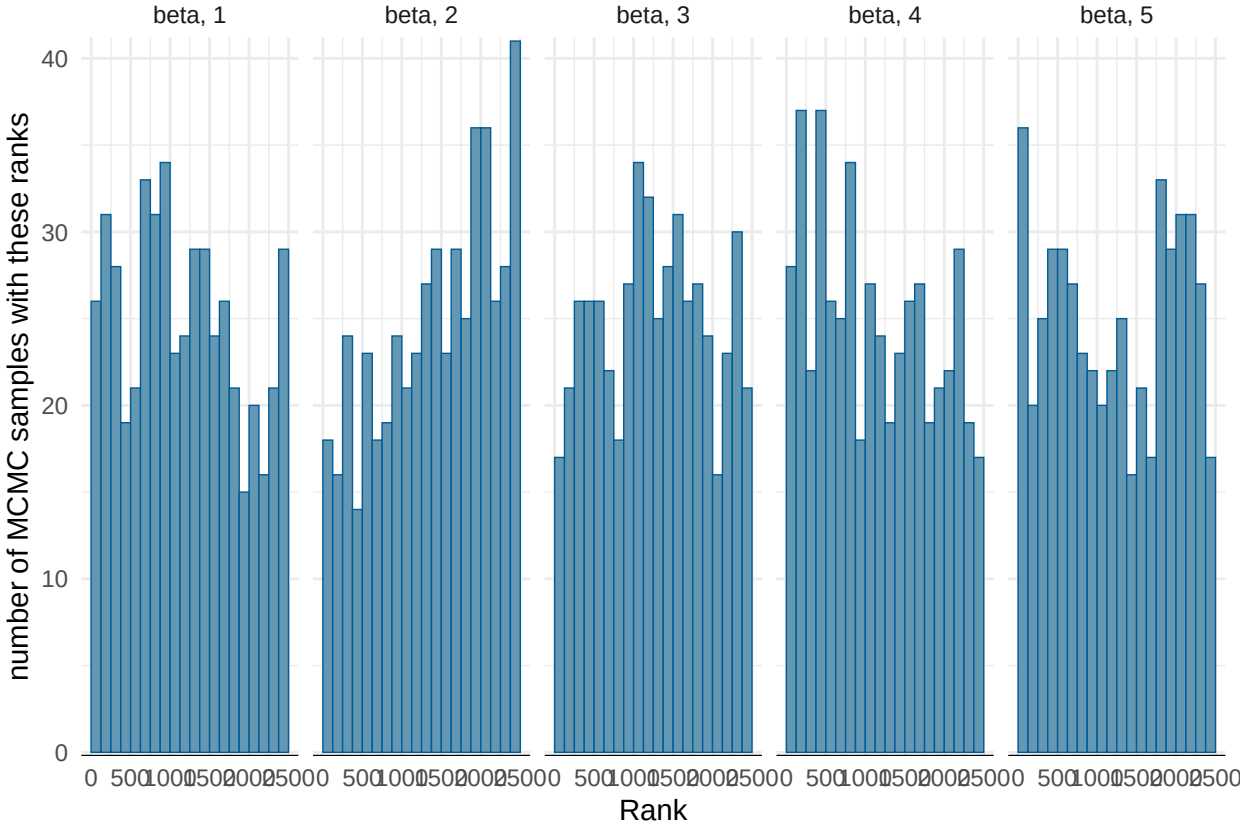



```
mcmc_trace(fit_exp.variational$draws("beta")) +
  theme_minimal() +
  labs(title = "Trace plot: beta (VI)")
```

```
mcmc_rank_hist(fit_simple_exp$draws("alpha")) + ylab("number of MCMC samples with these ranks") + theme_
```

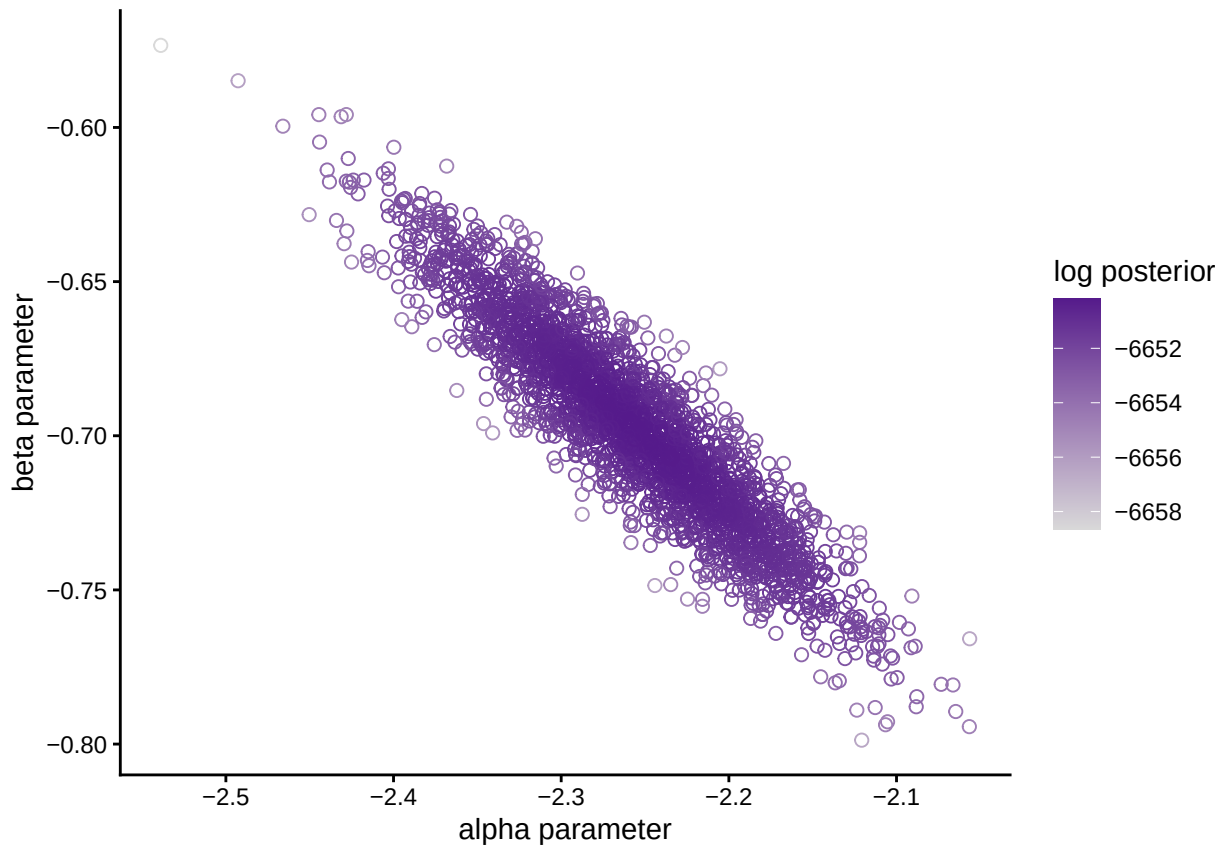



```
mcmc_rank_hist(fit_simple_exp$draws("beta")) + ylab("number of MCMC samples with these ranks") + theme_
```

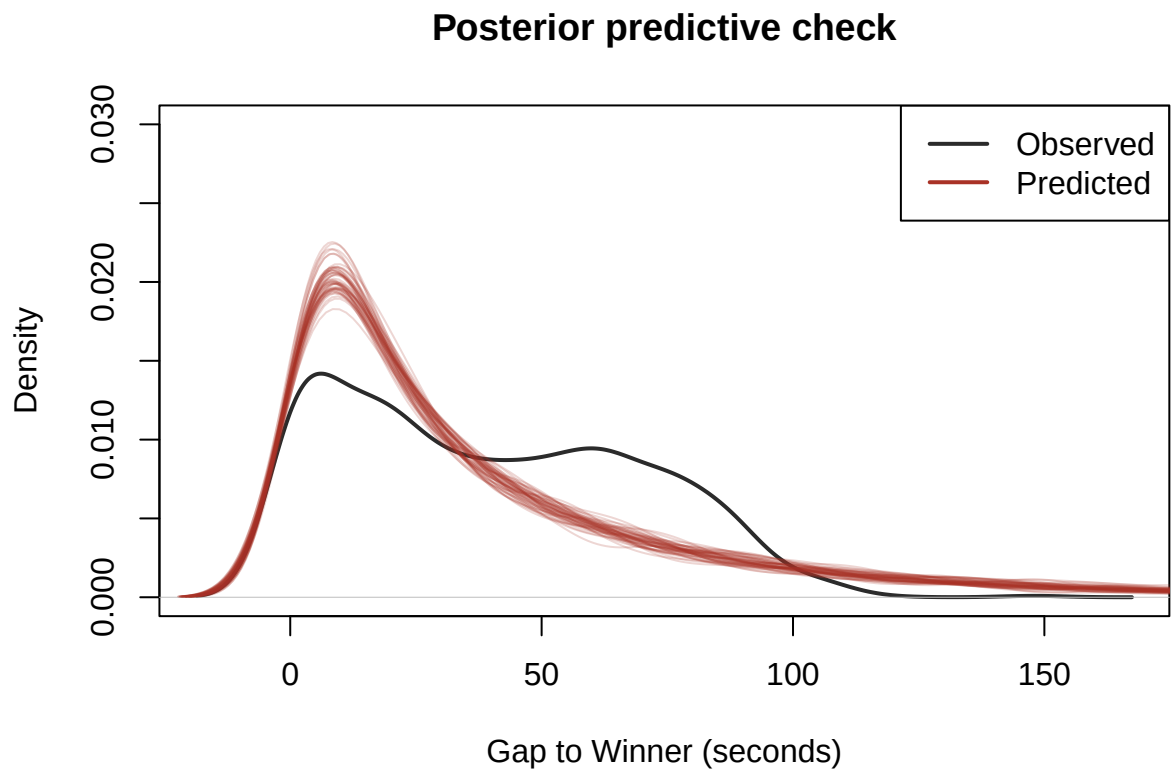



```
# mcmc_rank_hist(fit_exp.variational$draws("alpha")) + ylab("number of MCMC samples with these ranks") +
# mcmc_rank_hist(fit_exp.variational$draws("beta")) + ylab("number of MCMC samples with these ranks") +
```

```
# posterior parameter plot
# parameter values
simple_exp_draws_df <- fit_simple_exp$draws(format = "df")

ggplot(simple_exp_draws_df,
       aes(x = alpha, y = beta, color = lp__)) +
  geom_point(shape = 1, size = 2) +
  scale_color_gradient(low = "grey85", high = "purple4") +
  labs(
    x = "alpha parameter",
    y = "beta parameter",
    color = "log posterior"
  ) +
  theme_classic()
```



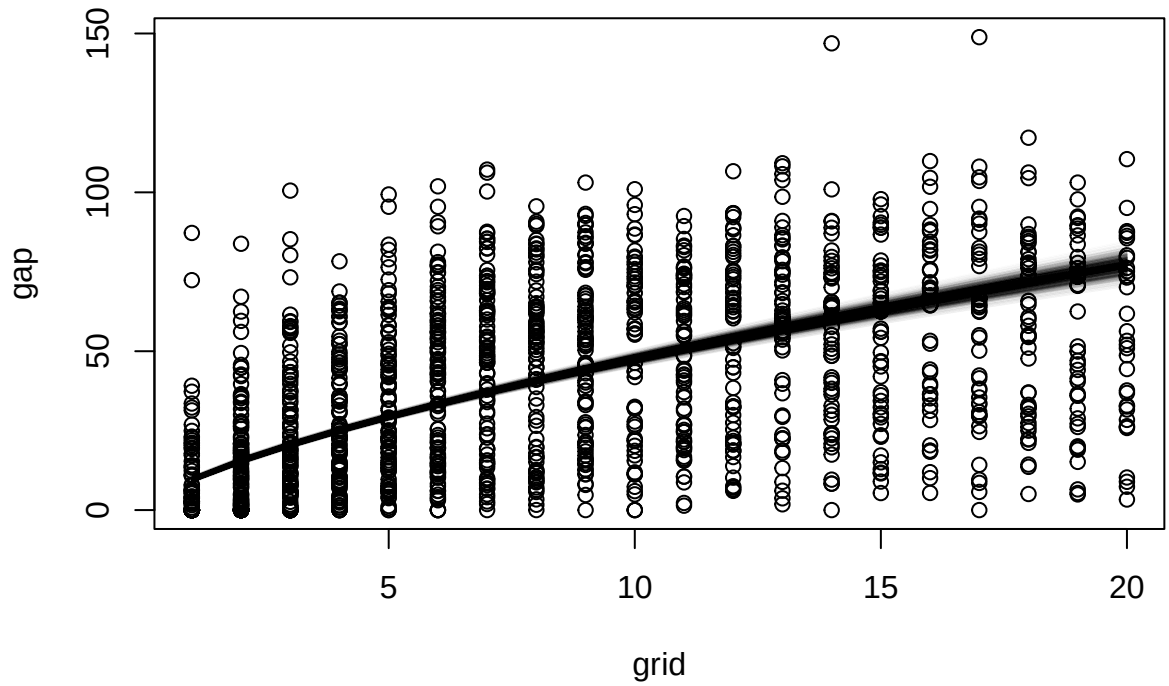

```r
# Posterior Predictive Check
plot(density(data$gap),
     main = "Posterior predictive check",
     xlab = "Gap to Winner (seconds)",
     col = "#2b2b2b", lwd = 2,
     ylim = c(0, 0.03))

for (i in sample(1:nrow(simple_exp_draws_df), 50)) {
  rate_pred <- exp(simple_exp_draws_df$alpha[i] +
                   simple_exp_draws_df$beta[i] * log(data$grid))
  gap_pred <- rexp(nrow(data), rate = rate_pred)
  lines(density(gap_pred), col = adjustcolor("#a93226", alpha.f = 0.2))
}

legend("topright",
       legend = c("Observed", "Predicted"),
       col = c("#2b2b2b", "#a93226"),
       lwd = 2)
```



The black line shows the observed gap distribution while the red lines show 50 datasets simulated from the posterior. The model captures the overall right-skewed shape but underestimates the spread in the middle range (30-100 seconds), suggesting the pooled Exponential model is an approximation. This motivates the hierarchical model which allows circuit-specific effects to better capture this variation.

```
# model against data
alpha_samples <- simple_exp_draws_df$alpha
beta_samples <- simple_exp_draws_df$beta

# normalize lp__ to get weights (analogous to norm_weights in your example)
lp <- simple_exp_draws_df$lp__
norm_weights <- exp(lp - max(lp))
norm_weights <- norm_weights / sum(norm_weights)

# grid of x values to plot over
xs <- seq(1, 20)

plot(data$grid, data$gap, xlab = "grid", ylab = "gap")

for (i in 1:nrow(simple_exp_draws_df)) {
  alpha <- alpha_samples[i]
  beta  <- beta_samples[i]

  # mean of exponential is 1/rate
  mean_gap <- exp(-(alpha + beta * log(xs)))

  lines(xs, mean_gap, col = rgb(0, 0, 0, alpha = norm_weights[i] * 20))
```

```
}
```




```
# TODO maybe do the same thing for VI?
```

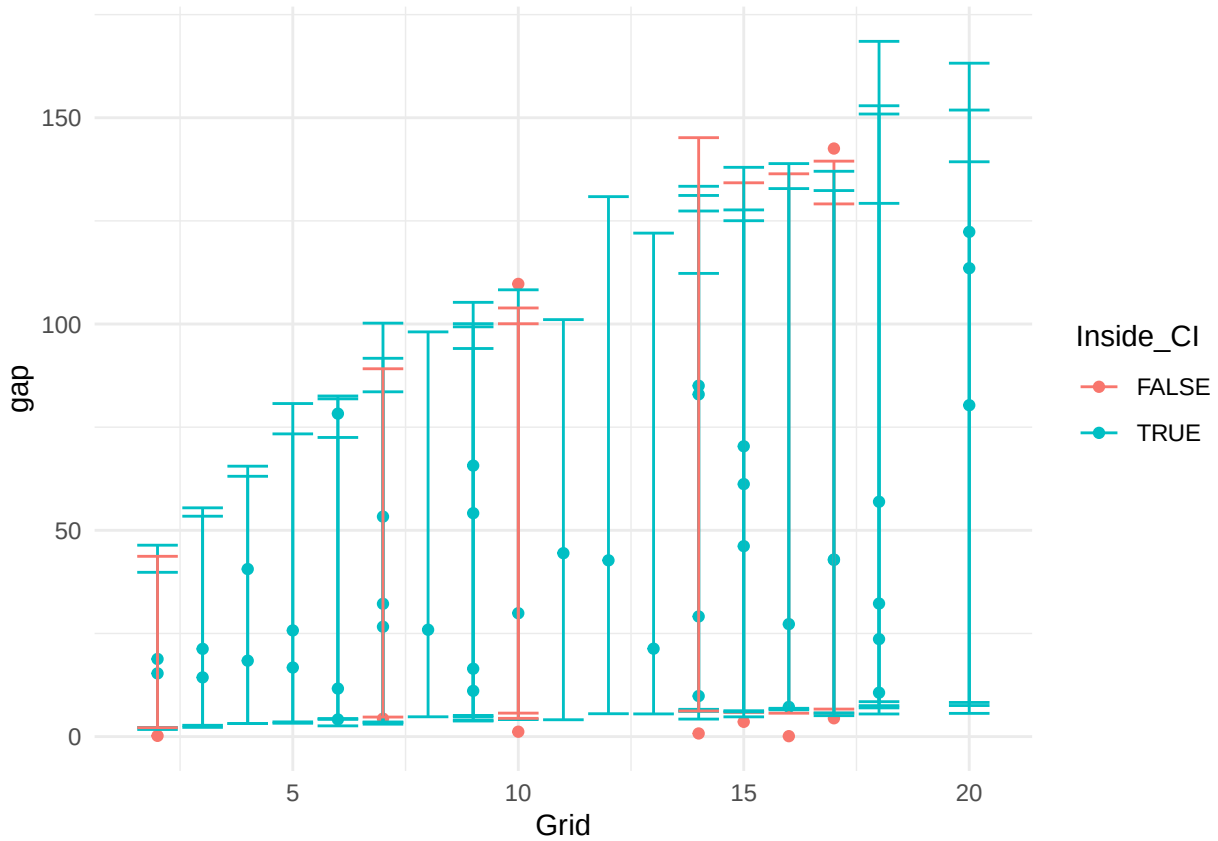
```
# ========= debugging ================
# calibration on simulated data

# sample parameters from their priors
# TODO

# parameters
alpha_mean = -2.5
alpha_sd = 1.5
beta_mean = -0.5
beta_sd = 0.75
```

```
# simulate_data: generates synthetic F1 gap data from the prior predictive
# distribution. Used to verify model correctness on known ground truth.

simulate_data = function() {
  grid = 1:20

  # sample parameters from their priors
  alpha_sim <- rnorm(1, alpha_mean, alpha_sd)
  beta_sim <- rnorm(1, beta_mean, beta_sd)
  log_grid = log(grid)
  rate = exp(alpha_sim + beta_sim * log_grid)
```

```r
  gap_sim <- as.vector(mapply(function(r) rexp(50, rate = r), rate))
  grid_rep <- rep(1:20, each = 50)

  return(data.frame(alpha = alpha_sim, beta = beta_sim, grid = grid_rep, gap = gap_sim))
}
```

```r
simulated_data = simulate_data()

validation_obs_debug <- sample(1:nrow(simulated_data), 50, replace = FALSE)
ci_limits_debug = matrix(0, nrow = 50, 2)

# WARNING: this takes a long time to run
for (i in 1:length(validation_obs_debug)) {
  i_test = validation_obs_debug[i]
  train_test_debug_dta = list(
    N = nrow(simulated_data) - 1,
    grid = simulated_data$grid[-i_test],
    gap = simulated_data$gap[-i_test],
    grid_pred = simulated_data$grid[i_test]
  )
  mcmc_results_debug = mod_simple_exp$sample(
    seed = 1,
    data = train_test_debug_dta,
    output_dir = "stan_out",
    refresh = 0,
    show_messages = FALSE,
    show_exceptions = FALSE,
    iter_warmup = 500,
    iter_sampling = 500,
    chains = 1
  )
  samples_debug = as.vector(mcmc_results_debug$draws("gap_pred"))
  ci_limits_debug[i, ] = quantile(samples_debug, ci_plims)
}
```

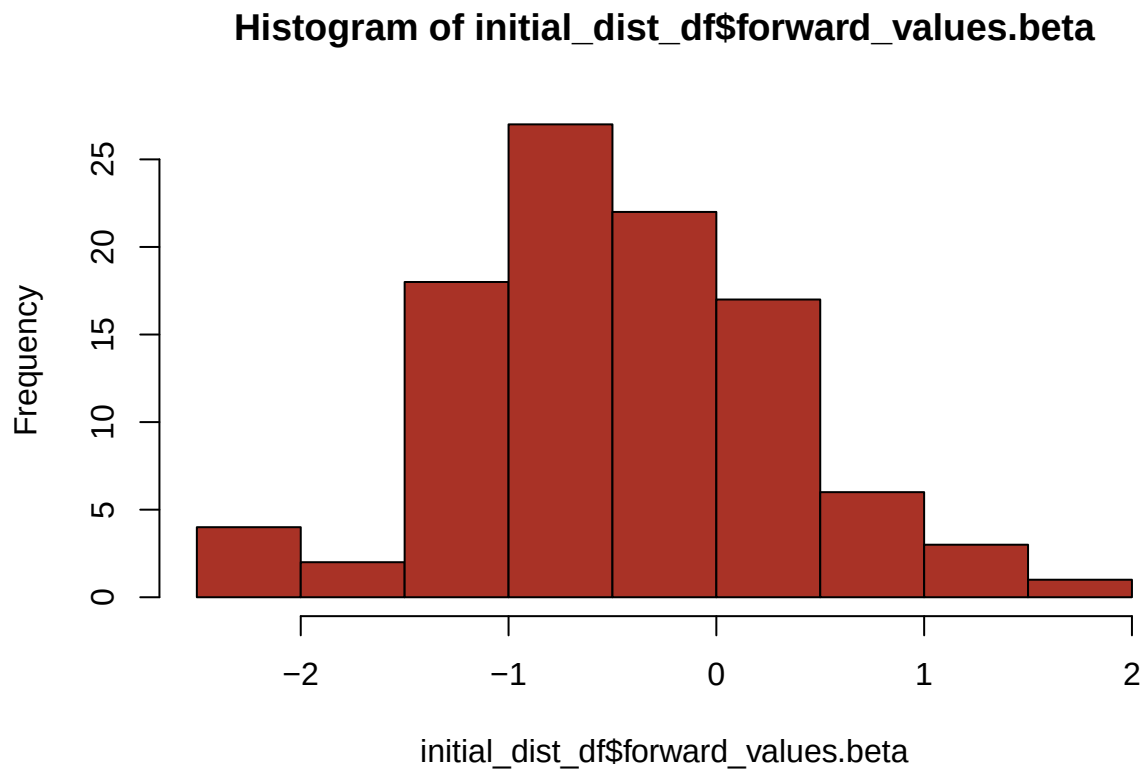
```r
# check coverage on synthetic data

merged_simulated_data = simulated_data[validation_obs_debug, ] %>%
  bind_cols(data.frame(CI_L = ci_limits_debug[, 1], CI_R = ci_limits_debug[, 2])) %>%
  mutate(Inside_CI = (gap >= CI_L & gap <= CI_R))

merged_simulated_data %>%
  ggplot(aes(x = grid, y = gap, ymin = CI_L, ymax = CI_R, color = Inside_CI)) +
  geom_point() +
  geom_errorbar() +
  theme_minimal() +
  labs(x = "Grid", y = "gap")
```




```r
mean(merged_simulated_data$Inside_CI)
```

```
## [1] 0.82
```

```r
# ========= exact invarianct test ================

# initial distribution
# 100 samples of alpha, beta for now
initial_dist_df = data.frame()
for (i in 1:100) {
  forward_values = simulate_data()
  initial_dist_df = rbind(initial_dist_df, data.frame(forward_values$alpha[1], forward_values$beta[1]))
}

hist(initial_dist_df$forward_values.beta, col = "#a93226")
```

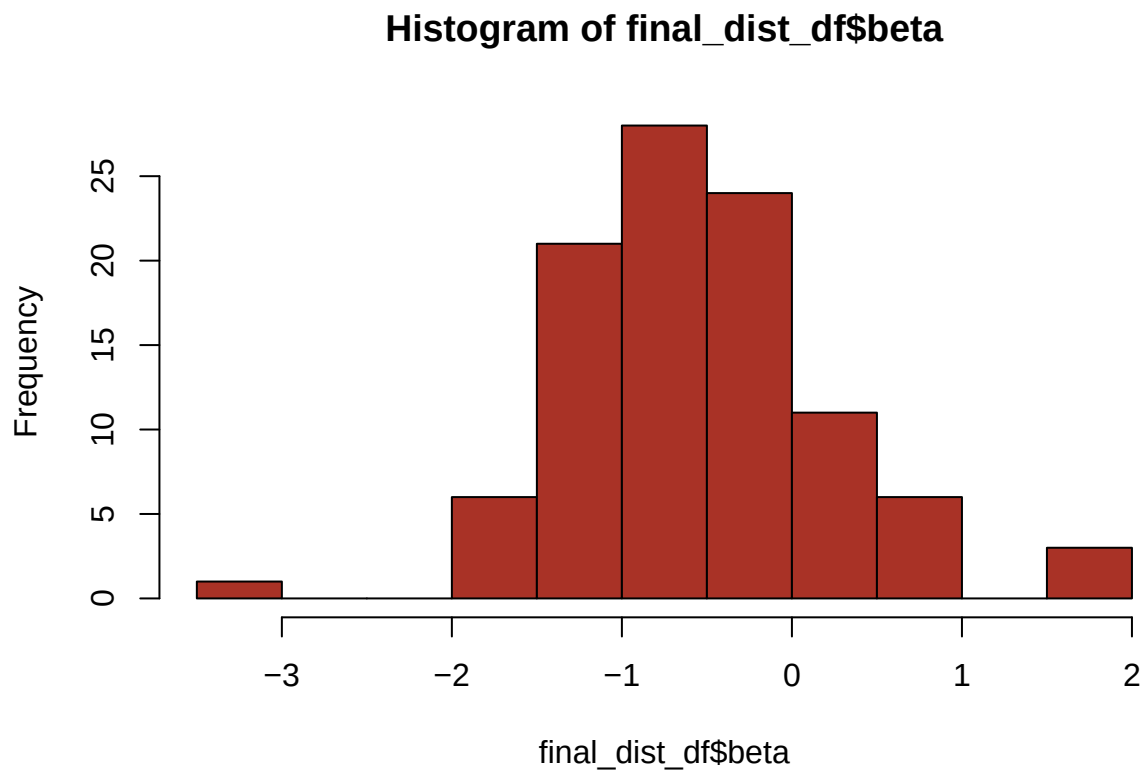



```r
# final distribution
final_dist_df = data.frame()
for (i in 1:100) {
  forward_values = simulate_data()
  dta = list(
    N = length(forward_values$grid),
    grid = forward_values$grid,
    gap = forward_values$gap,
    grid_pred = 10
  )
  curr_fit = mod_simple_exp$sample(
    seed = 1,
    data = dta,
    refresh = 0,
    show_messages = FALSE,
    show_exceptions = FALSE,
    output_dir = "stan_out",
    chains = 1,
    iter_warmup = 50,
    iter_sampling = 50,
    init = list(list(alpha = forward_values$alpha[1],
                     beta = forward_values$beta[1]))
  )
  final_params <- curr_fit$draws(format = "df") |>
    tail(1) |>
    subset(select = c(alpha, beta))
```

```r
  final_dist_df = rbind(final_dist_df, final_params)
}
```

```
## Warning: 1 of 1 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.
```

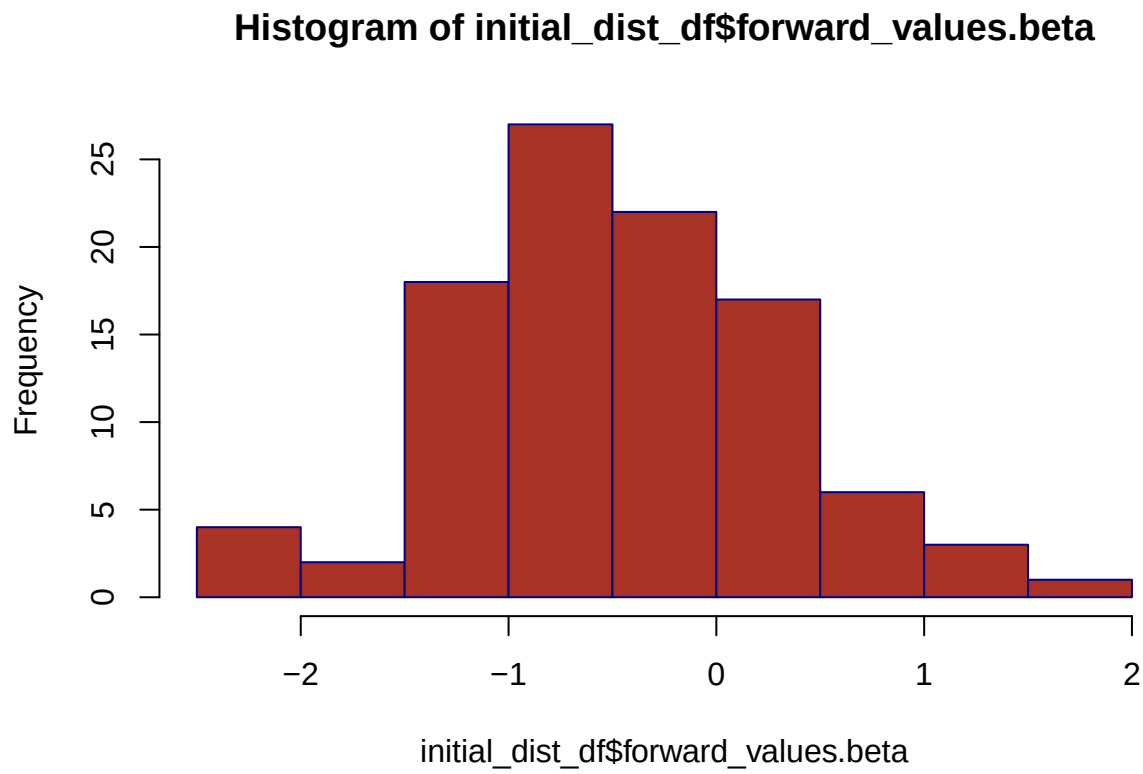
```r
hist(final_dist_df$beta, col = "#a93226")
```



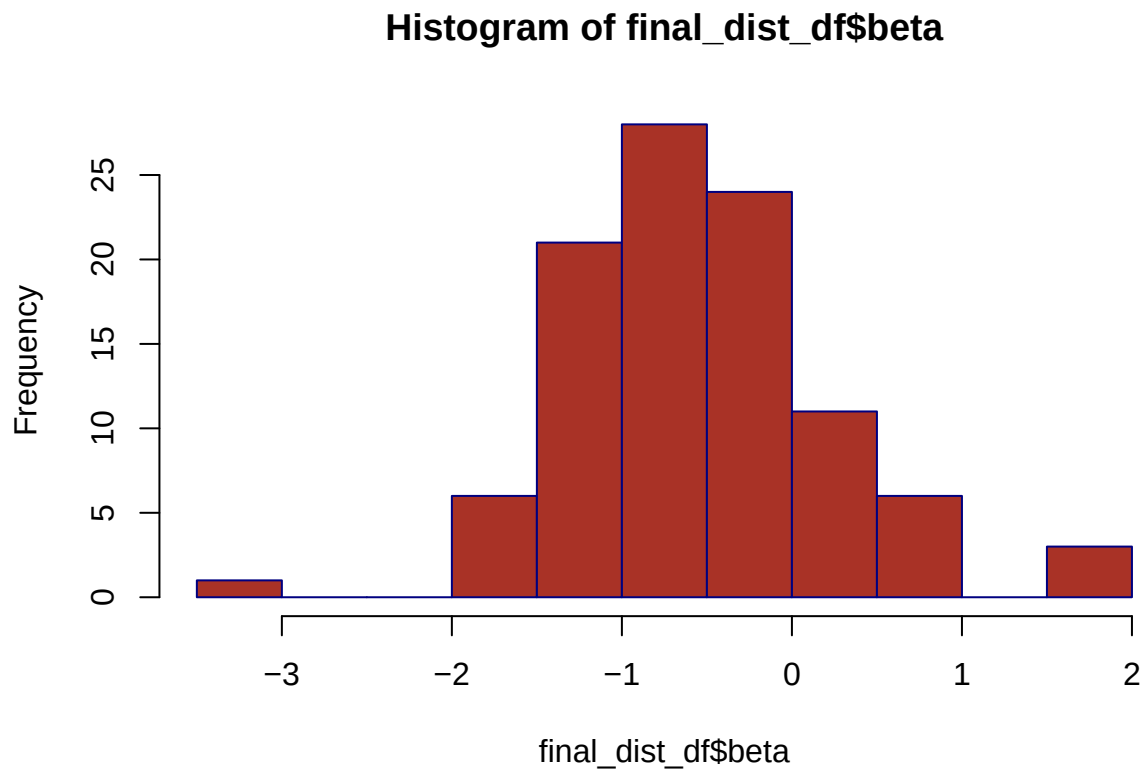

```r
ks.test(initial_dist_df$forward_values.beta, final_dist_df$beta)
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  initial_dist_df$forward_values.beta and final_dist_df$beta
## D = 0.08, p-value = 0.9062
## alternative hypothesis: two-sided
```

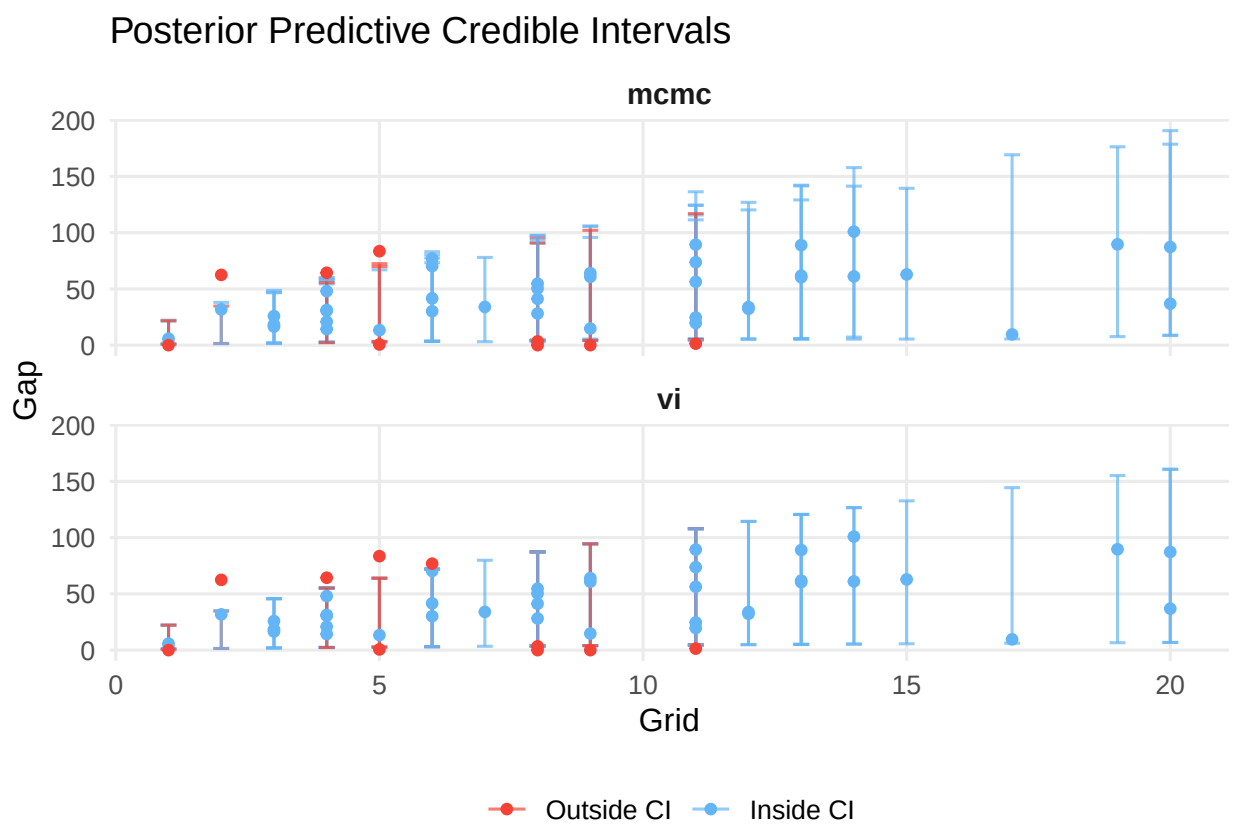
```r
hist(initial_dist_df$forward_values.beta, col = "#a93226",  border = "navy")
```




```
hist(final_dist_df$beta, col = "#a93226", border = "navy")
```

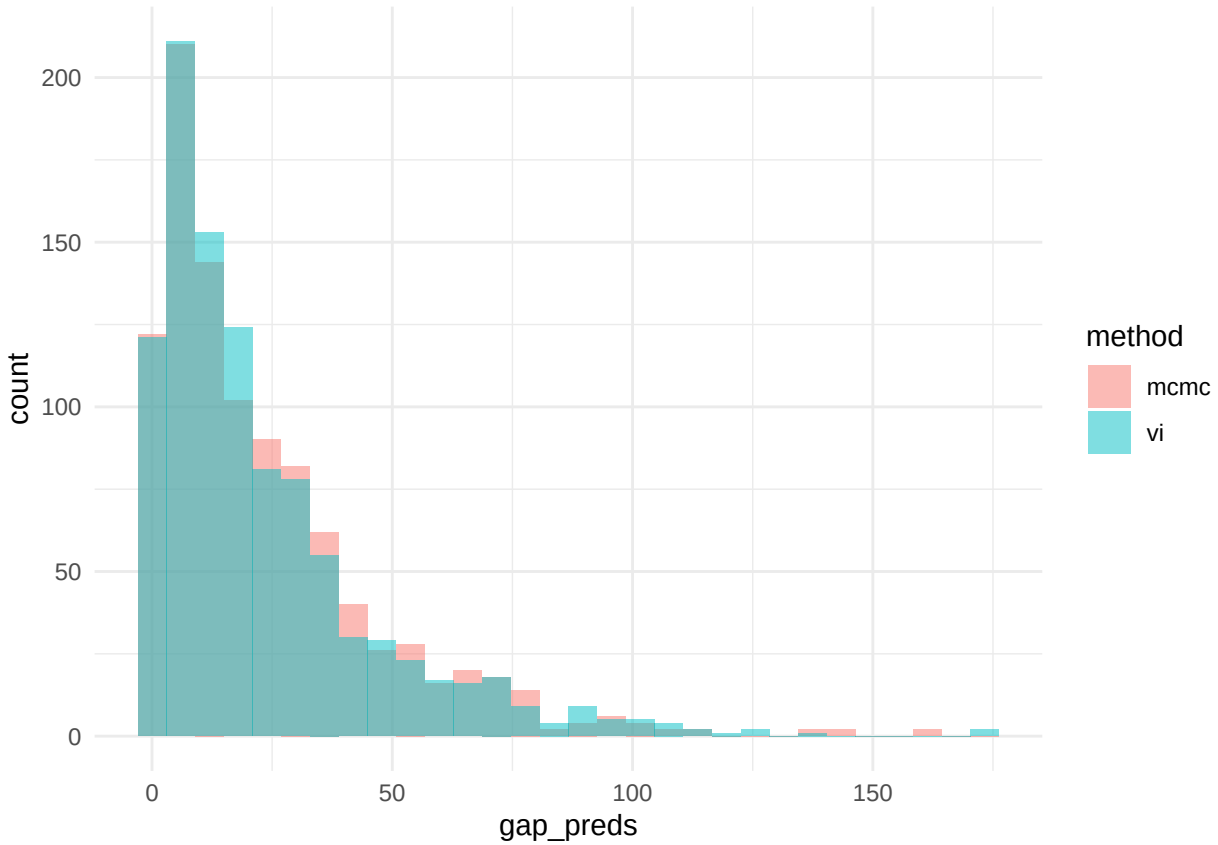



```r
# ========= MCMC vs VI ================
# compare credible intervals for mcmc and vi
merged_data_exp$method = "mcmc"
merged_data_exp_var$method = "vi"

combined_merged_data <- bind_rows(merged_data_exp, merged_data_exp_var)

combined_merged_data %>%
  ggplot(aes(x = grid, y = gap, ymin = CI_L, ymax = CI_R, color = Inside_CI)) +
  geom_errorbar(width = 0.3, alpha = 0.7) +
  geom_point(size = 1.5) +
  scale_color_manual(
    values = c("TRUE" = "#64B5F6", "FALSE" = "#F44336"),
    labels = c("TRUE" = "Inside CI", "FALSE" = "Outside CI")
  ) +
  facet_wrap(~ method, ncol = 1) +
  theme_minimal(base_size = 12) +
  theme(
    legend.position = "bottom",
    legend.title = element_blank(),
    strip.text = element_text(face = "bold", size = 11),
    panel.grid.minor = element_blank()
  ) +
  labs(title = "Posterior Predictive Credible Intervals",
       x = "Grid", y = "Gap")
```

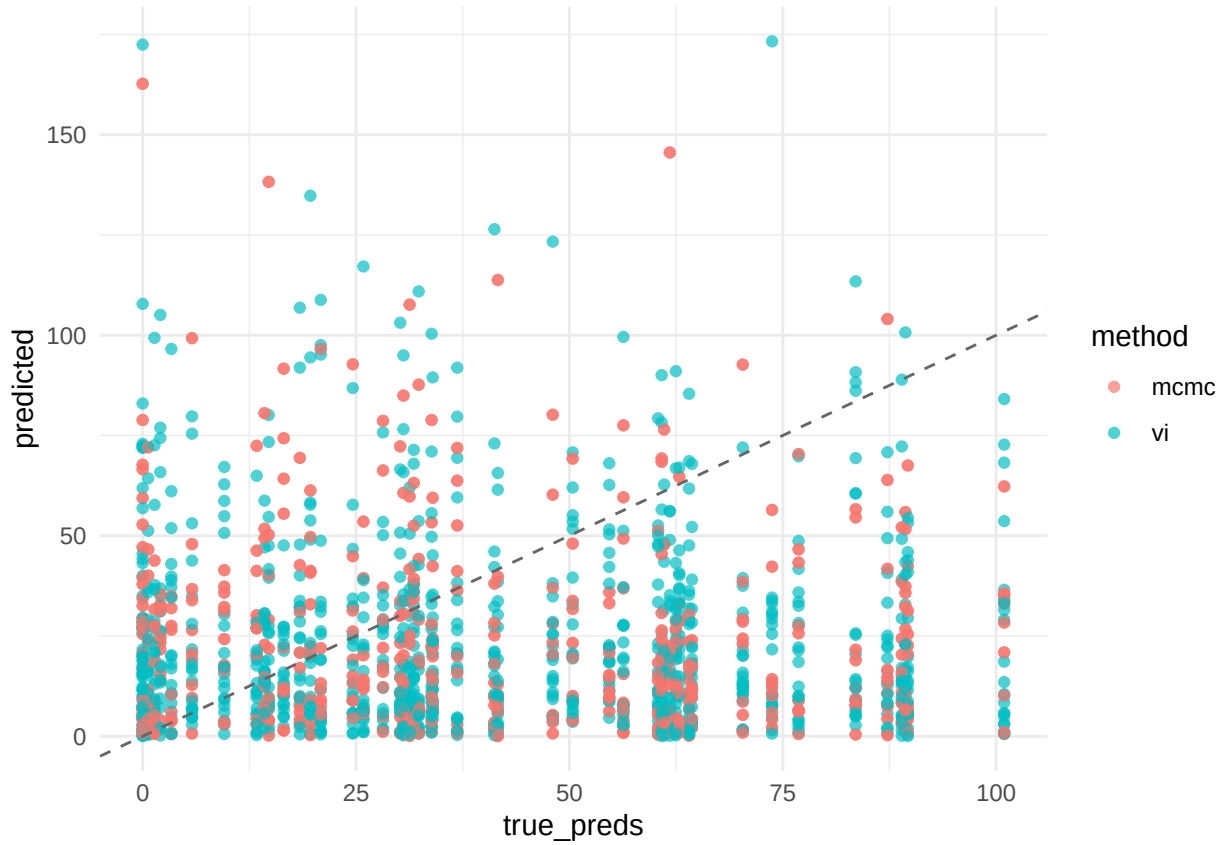



```r
# plot predicted gap vs actual gap for both methods
gap_preds_df = data.frame(
  mcmc = simple_exp_samples,
  vi = simple_exp_var_samples,
  true_preds = data[validation_obs, ]$gap
)
```

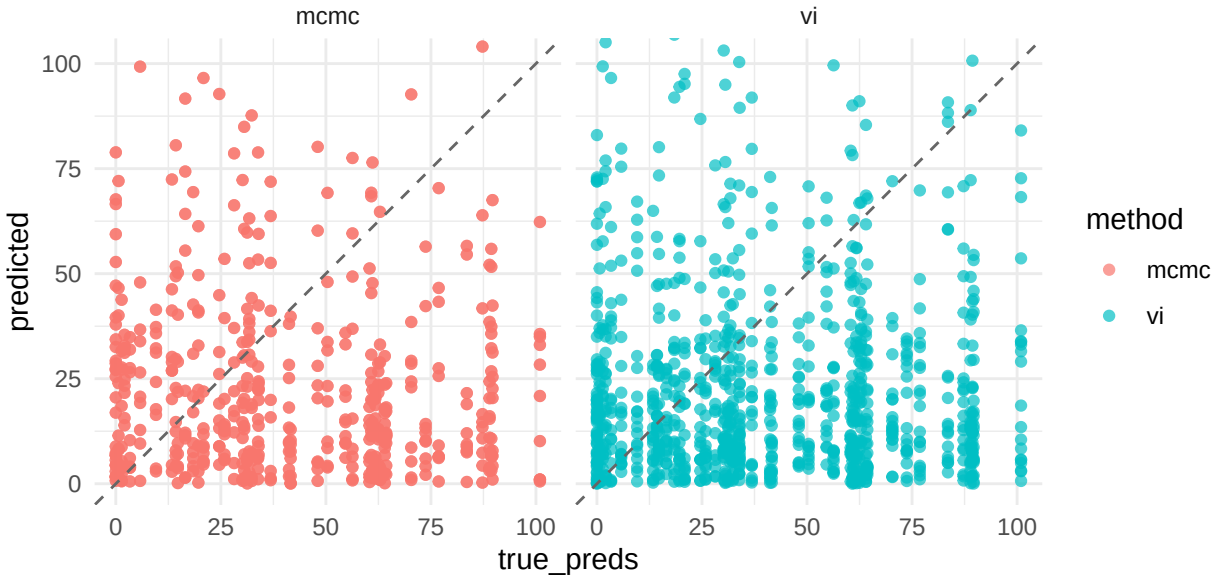
```r
gap_preds_df %>%
  select(mcmc, vi) %>%
  pivot_longer(cols = everything(), names_to = "method", values_to = "gap_preds") %>%
  ggplot(aes(x = gap_preds, fill = method)) +
  geom_histogram(alpha = 0.5, position = "identity", bins = 30) +
  theme_minimal()
```




```
gap_preds_df %>%
  pivot_longer(cols = c(mcmc, vi), names_to = "method", values_to = "predicted") %>%
  ggplot(aes(x = true_preds, y = predicted, color = method)) +
  geom_point(alpha = 0.7) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey40") +
  theme_minimal()
```




```
gap_preds_df %>%
  pivot_longer(cols = c(mcmc, vi), names_to = "method", values_to = "predicted") %>%
  ggplot(aes(x = true_preds, y = predicted, color = method)) +
  geom_point(alpha = 0.7) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey40") +
  coord_fixed(xlim = range(gap_preds_df$true_preds), ylim = range(gap_preds_df$true_preds)) +
  facet_wrap(~ method) +
  theme_minimal()
```


mcmc
vi
predicted
true_preds
0
25
50
75
100
method
mcmc
vi